Project 4 Task 1 – Disney Characters in Film Application
By Rathi Murugan (AndrewID: rmurugan)

Description:

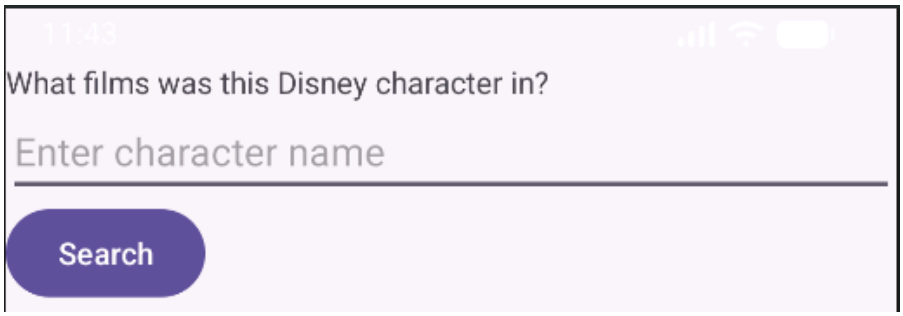
My application takes a character name from the user and returns information about that Disney character, including the films they appear in and an image. The application uses a third-party Disney API and logs all mobile interactions to a MongoDB database. A web-based dashboard is used to display analytics and logs.
Here is how my application meets the task requirements.

1. Implement a native Android application

The name of my native Android application project in Android Studio is:
Project4Android

a. Has at least three different kinds of views in your Layout

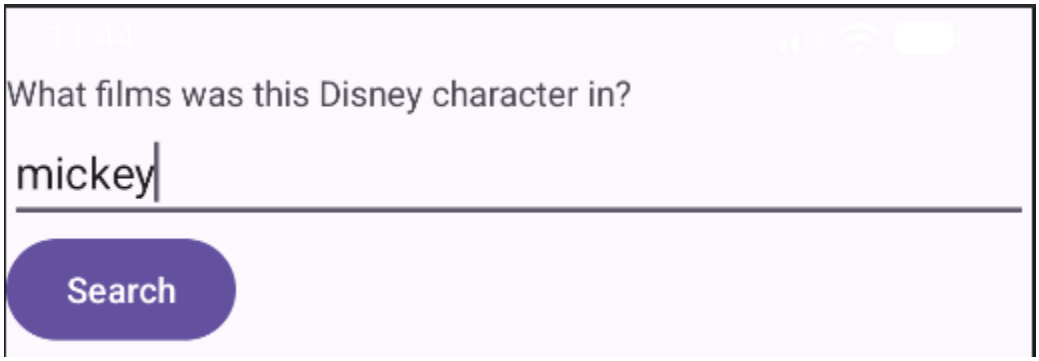
My application uses TextView, EditText, Button, and ImageView.
These are defined in content_main.xml and arranged in a vertical layout as shown below (with the image appearing after a successful search and shown in later screenshots).



---

b. Requires input from the user

The user enters a Disney character name into the EditText field and presses the Search button.



When testing, please be mindful that even though I implemented ranked searching to match the user's input most in the most flexible way with the most applicable character, some characters might require official names that your input might not match.

*This documentation was constructed with the help of the ChatGPT Model 5.3 LLM.

Project 4 Task 1 – Disney Characters in Film Application
By Rathi Murugan (AndrewID: rmurugan)

For safe testing, try these characters below:

- Gaston
- Ursula
- Rapunzel
- Mickey

---

c. Makes an HTTP request (using an appropriate HTTP method) to your web service

My application makes an HTTP GET request in GetCharacter.java.
The search() method constructs this request, sends it to the web service, and retrieves the response.

---

d. Receives and parses an XML or JSON formatted reply from the web service

The web service returns a JSON response of the form:

```
{
 "name": "Tiana",
 "imageUrl": "...",
 "films": [...]
}
```

The Android application parses this JSON using JSONObject and JSONArray in GetCharacter.java. Only the fields needed are returned.

---

e. Displays new information to the user

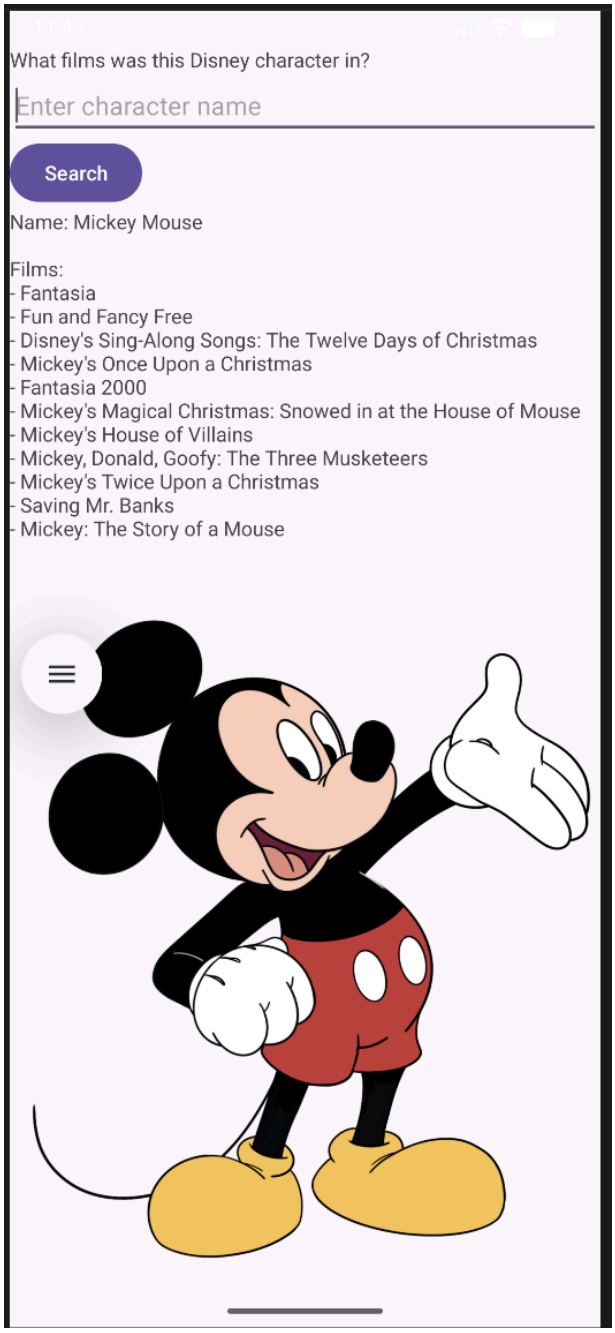
The application displays:

- the character name
- a list of films
- an image of the character

This is done in MainActivity.characterReady() and the output screen is shown below.

*This documentation was constructed with the help of the ChatGPT Model 5.3 LLM.

Project 4 Task 1 – Disney Characters in Film Application
By Rathi Murugan (AndrewID: rmurugan)



---

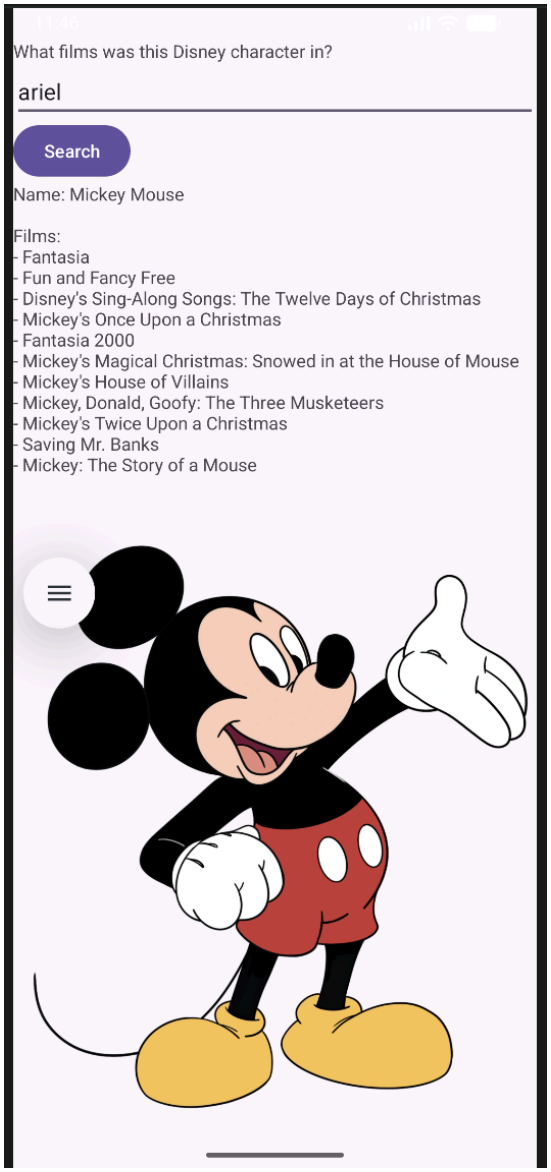
f. Is repeatable

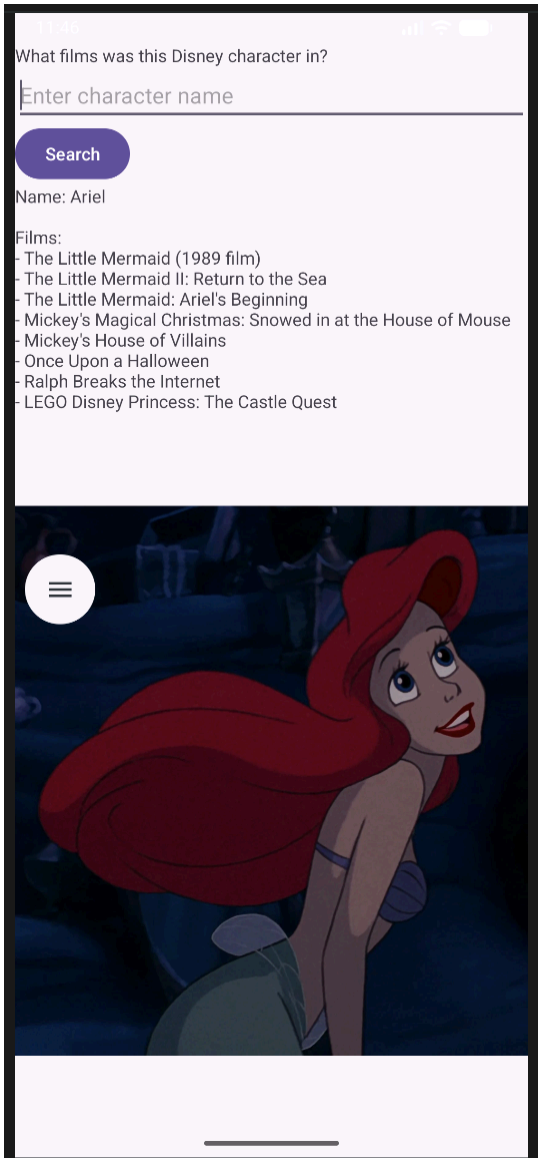
The user can repeatedly enter new character names and press Search without restarting the app.

The input field is cleared after each request (this process of being able to search a new character in a repeatable fashion is shown below in the series of screenshots).

*This documentation was constructed with the help of the ChatGPT Model 5.3 LLM.

Project 4 Task 1 – Disney Characters in Film Application
By Rathi Murugan (AndrewID: rmurugan)





---

2. Implement a web application, deployed to Codespaces

The web service is deployed and accessible through the Codespaces environment.

a. Using an HttpServlet to implement a simple API

The main servlet is:
DisneyCharacterServlet.java

- Model: DisneyCharacterModel.java
- Controller: DisneyCharacterServlet.java
- View (for dashboard): dashboard.jsp

---

*This documentation was constructed with the help of the ChatGPT Model 5.3 LLM.

b. Receives an HTTP request from the native Android application

The servlet receives HTTP GET requests at:
/getCharacter
It extracts the parameter name from the request.

---

c. Executes business logic appropriate to your application

The servlet calls DisneyCharacterModel, which:

- sends a request to the Disney API
- parses the JSON response
- returns candidate matches

The servlet then applies ranking logic to select the best match.

---

d. Replies to the Android application with a JSON formatted response

The servlet constructs a JSON response containing:

- character name
- image URL
- films array

This is sent back to the Android application.

---

4. Log useful information

For each mobile request, the application logs the following information:

- searchTerm: user input
- device: device making the request
- mobileEndpoint: endpoint accessed
- requestTimestamp: time request was received
- thirdPartyUrl: Disney API request URL
- thirdPartyStatusCode: HTTP response from API
- thirdPartyLatencyMs: time taken for API call
- candidateCount: number of results returned by API
- selectedCharacterName: final selected character
- filmCount: number of films returned
- imageReturned: whether an image was returned
- responseStatus: result of request (SUCCESS, ERROR, etc.)
- responseMessage: descriptive result message
- totalRequestLatencyMs: total request processing time

These were chosen to capture:

- user behavior
- API performance

*This documentation was constructed with the help of the ChatGPT Model 5.3 LLM.

- system performance
- result quality

---

5. Store the log information in a database

All logs are stored in MongoDB Atlas in the cloud.
Connection string (with shards):
mongodb://rmurugan:rmurugan123@ac-llcnnor-shard-00-02.l3iji5w.mongodb.net:27017,
ac-llcnnor-shard-00-01.l3iji5w.mongodb.net:27017,
ac-llcnnor-shard-00-00.l3iji5w.mongodb.net:27017/test?w=majority&retryWrites=true&tls=true&authMechanism=SCRAM-SHA-1
Logs are inserted using:
collection.insertOne(log)

---

6. Display operations analytics and full logs on a web-based dashboard

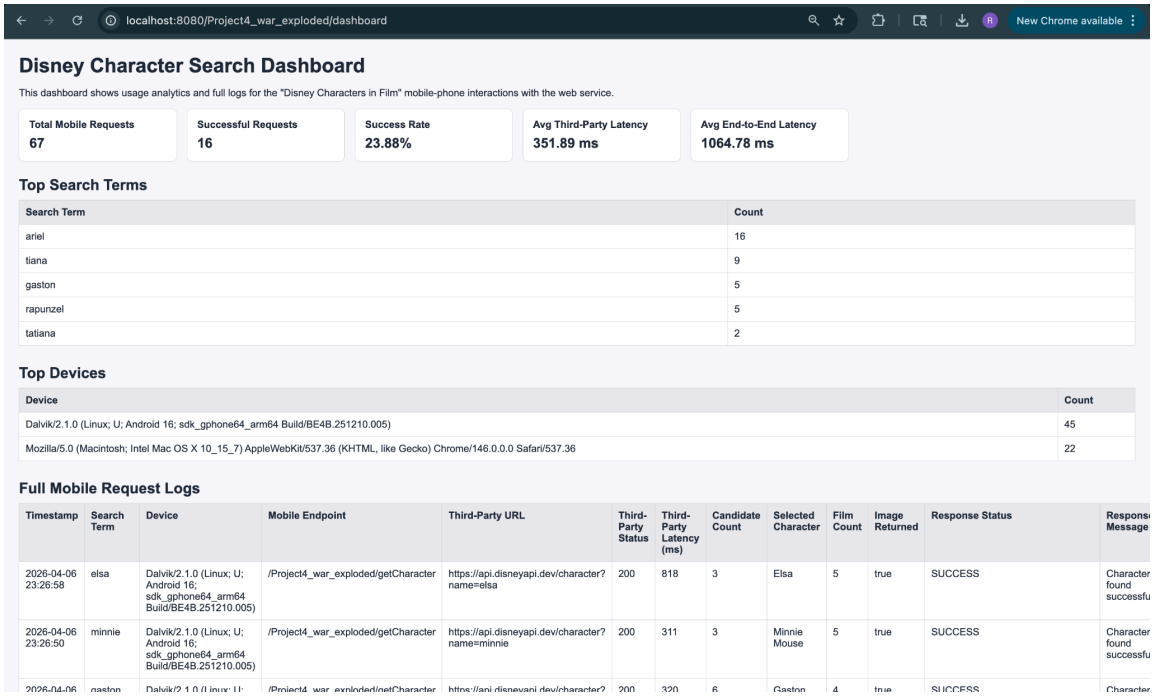
a. Unique URL:

The codespaces deployed dashboard unique URL link (at /dashboards as in DashboardServlet.java):
https://musical-memory-wr55rxrrgxvpf9gxw-8080.app.github.dev/dashboard

Please also note that I use a secondary servlet for the proposed of the analytics dashboard (this was approved by the Professor as a satisfactory approach).

*This documentation was constructed with the help of the ChatGPT Model 5.3 LLM.

Project 4 Task 1 – Disney Characters in Film Application
By Rathi Murugan (AndrewID: rmurugan)



b. Operation analytics displayed:

- Total number of requests
- Number of successful requests
- Success rate
- Average third-party API latency
- Average total request latency
- Top search terms
- Top devices used

Please also note that there are some older logs in my database that are also displayed at the very bottom of my full mobile request logs section that are remnants from my old testing and implementation and are no longer applicable.

c. Full Logs displayed:

All mobile interactions are shown in a formatted HTML table and include all tracked application log fields as mentioned above. The dashboard uses dashboard.jsp and displays data using

*This documentation was constructed with the help of the ChatGPT Model 5.3 LLM.

Project 4 Task 1 – Disney Characters in Film Application
By Rathi Murugan (AndrewID: rmurugan)

HTML tables for readability.

Full Mobile Request Logs

Timestamp	Search Term	Device	Mobile Endpoint	Third-Party URL	Third-Party Status	Third-Party Latency (ms)	Candidate Count	Selected Character	Film Count	Image Returned	Response Status	Re M
2026-04-14 12:27:43	aladdin	Dalvik/2.1.0 (Linux; U; Android 16; sdk_gphone64_arm64 Build/BE4B.251210.005)	/P4_Webservice_war_exploded/getCharacter	https://api.disneyapi.dev/character?name=aladdin	200	299	4	Aladdin	6	true	SUCCESS	Ch fo su
2026-04-14 12:27:22	jasmine	Dalvik/2.1.0 (Linux; U; Android 16; sdk_gphone64_arm64 Build/BE4B.251210.005)	/P4_Webservice_war_exploded/getCharacter	https://api.disneyapi.dev/character?name=jasmine	200	286	10	Jasmine	0	true	SUCCESS	Ch fo su
2026-04-14 12:27:15	moana	Dalvik/2.1.0 (Linux; U; Android 16; sdk_gphone64_arm64 Build/BE4B.251210.005)	/P4_Webservice_war_exploded/getCharacter	https://api.disneyapi.dev/character?name=moana	200	227	2	Moana	4	true	SUCCESS	Ch fo su
2026-04-14 12:27:11	mulan	Dalvik/2.1.0 (Linux; U; Android 16; sdk_gphone64_arm64 Build/BE4B.251210.005)	/P4_Webservice_war_exploded/getCharacter	https://api.disneyapi.dev/character?name=mulan	200	491	2	Fa Mulan	4	true	SUCCESS	Ch fo su
2026-04-14 12:26:47	mulan	Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0	/P4_Webservice_war_exploded/getCharacter	https://api.disneyapi.dev/character?name=mulan	200	393	2	Fa Mulan	4	true	SUCCESS	Ch fo su

---

7. System architecture

My webservice was successfully deployed using codespaces as required.

I deployed via the following private GitHub repository :
https://github.com/rmurugan58/project4-DisneyCharacterApp

Deployed dashboard link:
https://musical-memory-wr55rxrrgxvpf9gxw-8080.app.github.dev/dashboard

Deployed web service main backend API endpoint link (example with name=mulan):
https://musical-memory-wr55rxrrgxvpf9gxw-8080.app.github.dev/getCharacter?name=mulan

The system consists of:

- Android application (client)
- Java servlet web service (Codespaces)
- Third-party Disney API
- MongoDB Atlas database
- Web dashboard (browser)

The Android app sends requests to the servlet, which calls the Disney API, processes results, logs data to MongoDB, and returns a response. The dashboard reads from MongoDB to display analytics and logs.

*This documentation was constructed with the help of the ChatGPT Model 5.3 LLM.